PCET'S

Pimpri Chinchwad College of Engineering, Pune

Name: Soham Kulkarni **PRN:** 124B1F056 **Batch:** TY - A3

Course: Design and Analysis of Algorithm Laboratory

ASSIGNMENT NO. 05

1. Problem Statement

SwiftCargo specializes in delivering packages across multiple cities. Its transportation network is divided into stages representing warehouses, transit hubs and final delivery points. Each package must follow the most cost-efficient or fastest route from source to destination through the predefined stages.

• Model the network as a directed, weighted multistage graph with multiple intermediate stages.

• Implement an efficient optimal-route algorithm using dynamic programming.

• Support thousands of cities and routes using an adjacency-list representation.

• Analyze and optimize route selection using traffic conditions, weather delays and fuel estimates.

2. Course Objectives

• Apply dynamic programming to a real-world logistics problem.

• Study computational complexity, changing edge weights and batch delivery processing.

3. Course Outcomes

Model a staged network; find fastest and cheapest routes; update road conditions; process multiple delivery requests; and analyze time and space complexity.

4. Theory and Constraints

The network is a directed acyclic multistage graph. Every stage must contain at least one city. A road connects stage s to stage s+1, so any source-to-destination route visits one city in every stage. Source cities belong to the first stage and destination cities to the final stage. Multiple roads between the same pair of cities are supported and receive distinct road IDs.

4.1. Road Data and Optimization Modes

Each road stores its source, destination, base travel time, traffic delay, weather delay, estimated fuel use and open/closed status. Times are in minutes, fuel is in litres, and the user supplies a positive fuel price in INR per litre. Nonnegative decimal road values are accepted.

```
Fastest-mode weight = base time + traffic delay + weather delay
Cheapest-mode weight = estimated fuel litres * fuel price
```

All roads use the same units. The cheapest mode covers fuel expense; other charges such as tolls are outside this model. Fuel use is an entered estimate and can be revised when conditions change. Delays do not automatically change that estimate. The two objectives are optimized separately for each delivery request.

4.2. Dynamic Programming

```
best[destination] = 0
best[u] = minimum(weight(u, v) + best[v])
          over open roads reaching a reachable city v
```

Initialize other cities to infinity and process stages backwards. Store the best outgoing road for each city. Later-stage answers are already available when computing the answer for u. Following the saved road choices from the source reconstructs one optimal delivery route. If the source remains at infinity, no delivery route exists.

Correctness follows by backward induction: the destination cost is zero. Given optimal costs for the next stage, examining every usable outgoing road considers every possible first step from u. The smallest edge weight plus optimal remaining cost is therefore optimal for u.

4.3. Changing Conditions and Batch Requests

A menu updates a road by its input-order ID: change delays, fuel estimate, base time, or open/closed status. Closed roads are skipped. Fresh dynamic programming runs use the latest values. Updates are entered manually to simulate real-time changes; no live traffic or weather feed is connected.

A batch contains several source, destination and optimization-mode requests. Conditions stay fixed during a batch. Apply updates between batches, then rerun affected requests. Requests are independent; vehicle capacity and shared road reservations are outside this model.

5. Algorithm / Pseudocode

```
Read city count, stage count and city-stage assignments
Require a nonempty set of cities in every stage
Read fuel price and directed roads
Require each road to advance exactly one stage
Store roads by ID and adjacency lists of outgoing road IDs
Repeat until Exit is chosen:
    If Process Batch:
        Read number of requests Q
        For each request (source, destination, mode):
            Validate endpoints and mode
            Set all best values to infinity
            Set best[destination] = 0
            For stage s from last-1 down to 1:
                For each city u in stage s:
                    For each outgoing road r from u:
                        Skip closed roads / unreachable next cities
                        Compute time or fuel-cost weight for r
                        If weight + best[r.to] < best[u]:
                            Save new best[u] and next road ID
            If source unreachable: print no delivery route
            Otherwise print city route and road IDs
            Print total time, fuel use and fuel cost
    If Update Road:
        Read ID and replacement road conditions
        Validate and store new conditions
    If Show Roads:
        Display every road with current conditions
```

5.1. Complexity Analysis

Graph construction: O(V + E). The number of stages is at most V, and stage buckets avoid sorting city IDs. **One request:** O(V + E) time, including initialization and route reconstruction. **Batch of Q requests:** O(Q(V + E)); no cached results become stale after updates.

Total memory: O(V + E), including graph, stage buckets and DP arrays. Requests are read and processed one at a time, so memory does not grow with the batch size. **Road update:** O(1) using its road ID. **Display roads:** O(E).

Adjacency lists avoid the O(V squared) memory of an adjacency matrix. The iterative algorithm avoids recursive stack growth. Linear work per request makes sparse networks containing thousands of cities and routes practical. Total batch time still grows with the number of requests.

5.2. Input Handling and Tie Rules

Invalid stage assignments, skipped stages, invalid city/road IDs, negative values and invalid modes are rejected. Each numeric road value is limited to 10^12, and fuel price must be positive and at most 10^12. Invalid input terminates with an error. Equal-cost alternatives retain the first improving road in input order; road IDs identify parallel roads unambiguously.

6. Demo Network and Output

Illustrative network: 6 cities, 4 stages and 6 roads. City-stage assignments are [1, 2, 2, 3, 3, 4]. Source is city 1; destination is city 6. Fuel price is 100 INR/litre. Initially, all roads are open with zero traffic and weather delays.

Road ID	From	To	Base minutes	Fuel litres
1	1	2	5	3
2	1	3	7	1
3	2	4	5	3
4	3	5	7	1
5	4	6	5	3
6	5	6	7	1

6.1. Initial Batch: Fastest and Cheapest

```
Fastest delivery
Optimal route: 1 -> 2 -> 4 -> 6
Road IDs: 1 3 5
Total time: 15.00 minutes
Total fuel: 9.00 litres
Total fuel cost: 900.00 INR

Cheapest delivery
Optimal route: 1 -> 3 -> 5 -> 6
Road IDs: 2 4 6
Total time: 21.00 minutes
Total fuel: 3.00 litres
Total fuel cost: 300.00 INR
```

6.2. Real-Time Change Demonstration

Delay update: Road 3 receives 10 minutes of traffic delay and 2 minutes of weather delay. The first route now takes 27 minutes, so a fresh fastest request selects 1 -> 3 -> 5 -> 6 in 21 minutes.

Closure: Close road 4. A fresh request reroutes through 1 -> 2 -> 4 -> 6 in 27 minutes. Then close road 3 as well. Both branches are blocked, and the program prints: **No delivery route exists.**

7. Conclusion

Backward dynamic programming provides optimal staged routes for time or fuel cost. Road-ID updates support delays, closures and reopening. Batch requests reuse the current network and are solved independently in O(V + E) time each. The separate demo guide provides all input and menu steps; the C++ source is supplied separately from this report.